



Програмиране за мобилни интернет устройства

ПРОЦЕСИ И НИШКИ. ПАРАЛЕЛНА, АСИНХРОННА
РАБОТА И ОБРАБОТКА НА ИНФОРМАЦИЯ ВЪВ
ФОНОВ РЕЖИМ.

Концепция на паралелната обработка

- ▶ Последователно изпълнение – ограничения в хардуера
 - ▶ Един общ процес
 - ▶ Ниска производителност
- ▶ Съвременния хардуер
 - ▶ Многопроцесорен
 - ▶ Многоядрен
- ▶ Ефективност на изпълнение и производителност
 - ▶ Ефективно използване на хардуера
 - ▶ Използване на паралелни блокове на обработка

Процеси и нишки

- ▶ Процес
 - ▶ Съдържание на процеса – process context
 - ▶ Недостатък
- ▶ Нишка

Нишки. Същност, предимства и недостатъци

- ▶ „Лек“ процес – подпроцес в рамките на създаващият нишката процес
- ▶ Достъп до контекста на процеса
 - ▶ Бързодействие
 - ▶ Критични секции
 - ▶ Синхронизация – synchronized
 - ▶ Голямо влияние върху изпълнението и производителността
 - ▶ Ефективно използване
 - ▶ Принцип на най-малкия блок
- ▶ volatile

Проблеми при паралелна обработка

- ▶ Взаимна блокировка – deadlock
- ▶ Неконсистентност на данните (thread safe problem)
- ▶ Квази паралелност – гладни нишки

Нишки в Java

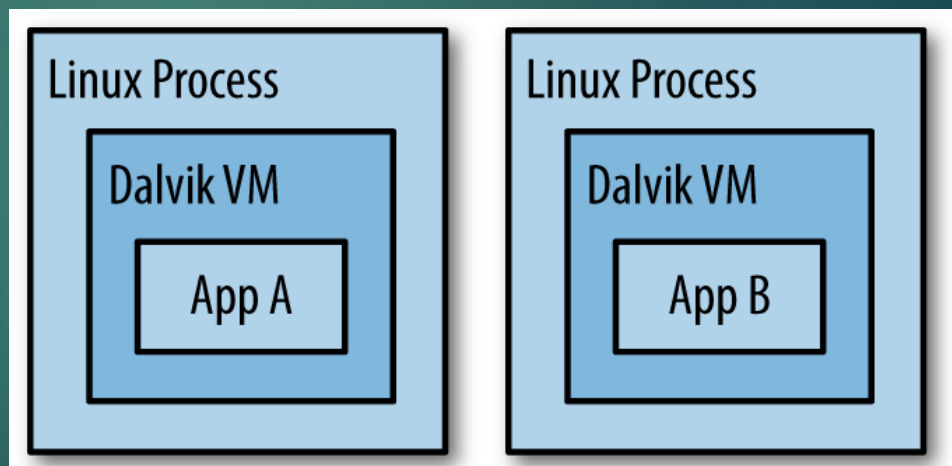
- ▶ Thread
 - ▶ run()
- ▶ Runnable
 - ▶ run()
- ▶ Callable
 - ▶ call()
- ▶ Executor (пул за нишки)
 - ▶ Една
 - ▶ Много
 - ▶ Една

Организация на работата на нишки в Java

- ▶ Приоритети
 - ▶ 1-10
 - ▶ 5
- ▶ `wait()`
- ▶ `notify()`, `notifyAll()`

Организация на процесите в Android

- ▶ Един процес, една нишка - "main" thread
- ▶ Атрибута android:process
 - ▶ <activity>, <service>, <receiver>, <provider>
 - ▶ <application>



Жизнен цикъл на процеса

- ▶ Йерархия на процесите

1. **Foreground process**

2. **Visible process**

- ▶ onPause()

3. **Service process**

- ▶ startService()

4. **Background process**

- ▶ onStop()

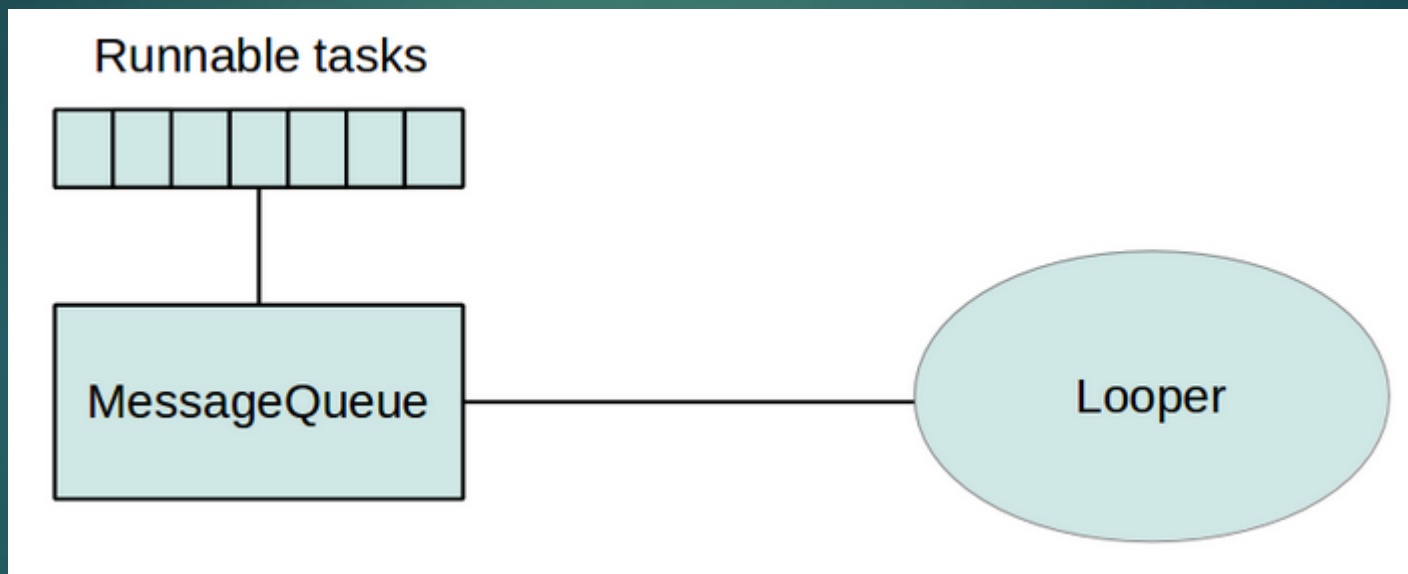
5. **Empty process**

Жизнен цикъл на процеса

- ▶ Определяне на нивото според компонентите
 - ▶ Видим компонент
 - ▶ Междупроцесна зависимост

Организация на процесите. Нишки

- ▶ "main" – диспечер на събития и изгледи



Организация на процесите. Нишки

- ▶ Недостатъци

- ▶ Andoid UI toolkit не е thread-safe

- ▶ "application not responding" (ANR)

1. Не блокирай UI thread

2. Не достъпвай Android UI toolkit извън контекста на UI thread

Работни нишки

- ▶ Worker threads
 - ▶ `Activity.runOnUiThread(Runnable)`
 - ▶ `View.post(Runnable)`
 - ▶ `View.postDelayed(Runnable, long)`

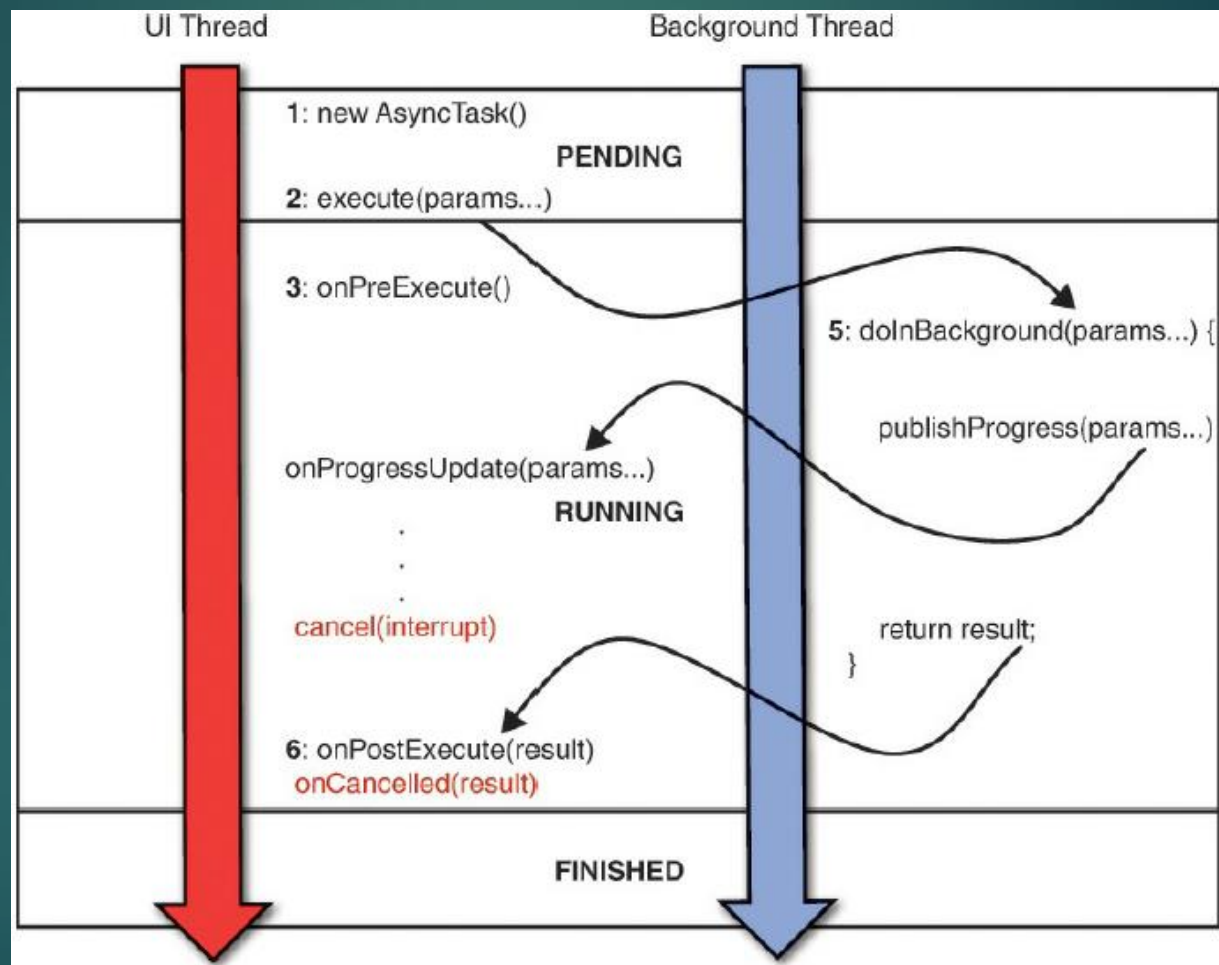
Работни нишки

```
public void onClick(View v) {  
    new Thread(new Runnable() {  
        public void run() {  
            final Bitmap bitmap = processBitMap("image.png");  
            imageView.post(new Runnable() {  
                public void run() {  
                    imageView.setImageBitmap(bitmap);  
                }  
            });  
        }  
    }).start();  
}
```

Класът **AsyncTask**

- ▶ Асинхронна работа на UI интерфейса
 - ▶ Наследяване на `AsyncTask`
 - ▶ Имплементиране на `(doInBackground(...))` и `(onPostExecute())`

Класът AsyncTask



Класът AsyncTask

- ▶ Params - вида на параметрите изпращани на задачата за изпълнение
- ▶ Progress
- ▶ Result - типа на резултата
- ▶ Незадължителност на параметрите
 - ▶ `private class MyTask extends AsyncTask<Void, Void, Void> { ... } - varargs`
- ▶ UI thread
 1. `onPreExecute()`
 2. **`doInBackground(Params...) !!!`**
 - ▶ Нова инстанция
 - ▶ `publishProgress(Progress...)`
 3. `onProgressUpdate(Progress...)`
 4. `onPostExecute(Result)`

Класът AsyncTask

```
private class DownloadFilesTask extends AsyncTask<URL, Integer, Long> {  
    protected Long doInBackground(URL... urls) {  
        int count = urls.length;  
        long totalSize = 0;  
        for (int i = 0; i < count; i++) {  
            totalSize += Downloader.downloadFile(urls[i]);  
            publishProgress((int) ((i / (float) count) * 100));  
  
            if (isCancelled()) break;  
        }  
        return totalSize;  
    }  
  
    protected void onProgressUpdate(Integer... progress) {  
        setProgressPercent(progress[0]);  
    }  
  
    protected void onPostExecute(Long result) {  
        showDialog("Downloaded " + result + " bytes");  
    }  
}
```

Класът `AsyncTask`

- ▶ Стартиране на задача
 - ▶ **new** `DownloadFilesTask().execute(url1, url2, url3)`

Прекратяване на задача

- ▶ `cancel(boolean), isCancelled()`
- ▶ Промени в конфигурацията в реално време

Класът `AsyncTask`

- ▶ Винаги през UI нишката
- ▶ Стартиране само веднъж
- ▶ Гарантира синхронизация между отделните си методи

Проблеми при работа с **AsyncTask**

- ▶ Проблеми с паралелен характер
 - ▶ Достъп до ресурс от UI нишката
- ▶ Проблем при жизнения цикъл
 - ▶ Промяна на конфигурация

Handler

- ▶ Взаимодействие с Looper
- ▶ Принцип на шината
- ▶ Абонамент за съобщения
 - ▶ Опашка за съобщения

Thread-safe МЕТОДИ

- ▶ При необходимост от множествен достъп
 - ▶ bound service

КЛАСИЧЕСКИ АСИНХРОНЕН ПОДХОД

- ▶ принцип на callback методи
 - ▶ механизъм, при който една функция (callback) се предава като аргумент на друга функция или метод, за да бъде извикана по-късно, когато определено събитие настъпи или операция завърши
 - ▶ позволяват **асинхронно програмиране**, при което дадена операция може да продължи, без да блокира текущия поток или нишка

Основни стъпки в работата на callback методите

- ▶ Регистрация на callback метода:
 - ▶ Callback методът се предава като аргумент на друга функция/метод
 - ▶ `performTask(callback);`
- ▶ Изпълнение на задача:
 - Основната функция започва изпълнението на операцията (например бавна мрежова заявка).
- ▶ Извикване на callback:
 - След завършване на операцията или при настъпване на събитие, callback методът се извиква.
 - Callback-ът получава резултата от операцията (ако има такъв) като аргумент.

Пример за работа на callback метод в Java

```
public interface Callback {  
    void onSuccess(String result);  
    void onError(Exception e);  
}
```

Пример за работа на callback метод в Java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class TaskRunner {

    private final ExecutorService executor =
        Executors.newSingleThreadExecutor();

    public void performTask(Callback callback) {
        executor.submit(() -> {
            try {
                // Симулиране на бавна задача
                Thread.sleep(2000);

                String result = "Task completed successfully";

                // Уведомяване за успех
                callback.onSuccess(result);
            } catch (Exception e) {
                // Уведомяване за грешка
                callback.onError(e);
            }
        });
    }
}
```

Пример за работа на callback метод в Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Task started...");  
        TaskRunner runner = new TaskRunner();  
    }  
}  
  
runner.performTask(new Callback() {  
    @Override  
    public void onSuccess(String result) {  
        System.out.println("Success: " + result);  
    }  
  
    @Override  
    public void onError(Exception e) {  
        System.err.println("Error: " + e.getMessage());  
    }  
});
```

Пример за работа на callback метод в Java

1. Регистрация:

- ▶ Callback се предава като аргумент на метода `performTask`.
- ▶ В този пример, предоставеният callback има две дефинирани операции: `onSuccess` и `onError`.

2. Изпълнение:

- ▶ Методът `performTask` изпълнява бавна задача във фонов поток с помощта на `ExecutorService`.

3. Уведомяване:

- ▶ Ако задачата приключи успешно, `callback.onSuccess(result)` се извиква.
- ▶ Ако възникне грешка, `callback.onError(e)` се извиква.

4. Реакция:

- ▶ В `main` функцията, callback-ът обработва резултата или грешката и извежда съответното съобщение.

Силни страни на callback методите

▶ Асинхронност:

- Позволяват изпълнение на бавни задачи, без да блокират главния поток или нишка.

▶ Гъвкавост:

- Позволяват изпълнение на различни действия в зависимост от резултата на операцията.

▶ Интеграция:

- Callback методите са широко използвани в мрежови операции, анимации, бази данни и много други контексти.

Ограничения на callback методите

- ▶ Callback Hell:
 - ▶ При сложни задачи с множество вложени callback-и кодът може да стане труден за четене и поддръжка.
 - ▶ Трудност при управление на грешки:
 - ▶ Грешките трябва да се управляват ръчно във всеки callback.
- ▶ Непоследователен код:
 - ▶ Callback методите могат да разделят изпълнението на логиката на множество места, което прави кода по-труден за следене.

Какво представлява "Callback Hell"?

- ▶ когато имаме много вложени callback функции, което води до труден за четене и поддръжка код

```
performTask(result -> {  
  anotherTask(result, nextResult -> {  
    finalTask(nextResult, finalResult -> {  
      System.out.println("Final result: " + finalResult);  
    }, error -> {  
      System.err.println("Error in final task");  
    });  
  }, error -> {  
    System.err.println("Error in another task");  
  });  
}, error -> {  
  System.err.println("Error in performTask");  
});
```

Алтернативи на callback МЕТОДИТЕ

► Promises/Futures:

- Позволяват управление на асинхронни задачи чрез "обещания" за бъдещ резултат.
- Пример: `CompletableFuture` в Java.

► Kotlin Coroutines:

- Използват `suspend` функции, което прави кода по-четивен и лесен за поддръжка.

► RxJava:

- Осигурява реактивно програмиране с потоци от данни.

Kotlin coroutines

- ▶ леки изпълнителни единици, използвани за асинхронно програмиране
- ▶ могат да бъдат суспендирани и възобновени, без да блокират текущата нишка
- ▶ интегрирани в Kotlin

Как работят корутините?

► Лекота:

- Корутините не създават нова нишка. Вместо това те използват съществуващи нишки и могат да споделят една нишка с множество корутини.

► Блокиране и възобновяване:

- Корутината може да бъде временно "замразена" (suspend), докато чака резултат, и след това "събудена" и продължена от същата точка, когато резултатът е готов.

► Асинхронност:

- Позволяват писане на асинхронен код по **синхронен начин**, което го прави по-четим и лесен за поддръжка.

Ключови елементи на корутините

► **suspend** функции:

- Функции, които могат да бъдат блокиране и възобновени. Използват се само в корутинен контекст.
- ```
suspend fun fetchData(): String {
 delay(1000L) // Суспендира корутината за 1 секунда
 return "Data fetched"
}
```

# Ключови елементи на корутините

## ▶ Корутинен билдър:

- Стартира нова корутина.
- Основни билдъри в Kotlin:
  - **launch**: Стартира корутина, която не връща резултат.
  - **async**: Стартира корутина, която връща резултат чрез Deferred.

## ▶ CoroutineScope:

- Определя обхвата на корутината и контролира нейния жизнен цикъл.
- Примери: GlobalScope, runBlocking.

## ▶ Диспечер (Dispatcher):

- Контролира на коя нишка или контекст ще се изпълни корутината.
  - **Dispatchers.Main**: Главната (UI) нишка.
  - **Dispatchers.IO**: За операции с вход/изход (мрежови заявки, четене/запис в база данни).
  - **Dispatchers.Default**: За тежки изчисления.

# Пример за корутина в Kotlin

```
import kotlinx.coroutines.*

fun main() = runBlocking {
 println("Start: ${Thread.currentThread().name}")

 // Стартира корутина
 val job = launch {
 println("Running in: ${Thread.currentThread().name}")
 delay(2000L) // Суспендира корутината за 2 секунди
 println("Completed in: ${Thread.currentThread().name}")
 }

 println("Waiting...")
 job.join() // Изчаква корутината да завърши

 println("End: ${Thread.currentThread().name}")
}
```

# Пример за корутина в Kotlin

## ▶ **runBlocking:**

- Създава корутинен обхват, който блокира текущата нишка (в случая `main`), докато всички корутини в него завършат.

## ▶ **launch:**

- Стартира нова корутина за изпълнение на асинхронна операция.

## ▶ **delay:**

- Блокира корутината за даден период от време, без да блокира нишката.

## ▶ **job.join():**

- Изчаква текущата корутина да завърши.



# Пример за паралелно изпълнение с `async`

```
import kotlinx.coroutines.*

fun main() = runBlocking {
 val timeStart = System.currentTimeMillis()

 val task1 = async { performTask(1, 2000L) }
 val task2 = async { performTask(2, 3000L) }

 println("Result 1: ${task1.await()}")
 println("Result 2: ${task2.await()}")

 val timeEnd = System.currentTimeMillis()
 println("Total time: ${timeEnd - timeStart} ms")
}

suspend fun performTask(taskId: Int, delayTime: Long): String {
 delay(delayTime)
 return "Task $taskId completed"
}
```

# Пример за паралелно изпълнение с `async`

## 1. `async`:

- ▶ Стартира паралелно изпълнение на две корутини (`task1` и `task2`).
- ▶ Връща обект от тип `Deferred`, който може да се използва за извикване на `await()` и получаване на резултата.

## 2. `await()`:

- ▶ Изчаква резултата от асинхронната операция.

# Предимства на корутините

- ▶ **Асинхронно програмиране без сложност:**
  - Кодът изглежда линейно, въпреки че е асинхронен.
- ▶ **Не блокират нишките:**
  - Корутините суспендират своето изпълнение, без да блокират текущата нишка.
- ▶ **Мащабируемост:**
  - Една нишка може да управлява хиляди корутини.
- ▶ **Гъвкавост:**
  - Позволяват лесно превключване между контексти (например от `Dispatchers.IO` към `Dispatchers.Main`).

# Недостатъци на корутините

## ► Изискват коректен жизнен цикъл:

- Ако корутината не бъде отменена правилно, тя може да причини течове на памет (memory leaks).

## ► Не са магически:

- Лошо проектирани корутини или прекомерно създаване на корутини могат да доведат до проблеми с производителността.

## ► Обучение:

- Разработчиците трябва да научат основите на корутините и как да ги използват правилно.

# Кога да използваме корутини?

## ▶ Асинхронни задачи:

- Мрежови операции, достъп до база данни, работа с файлове.

## ▶ Паралелни операции:

- Изчисления, които могат да се изпълняват паралелно.

## ▶ Анимации и UI операции:

- Работа с анимации или обновяване на UI в Android.